LangGraph Mastery

Developing LLM agents with LangGraph

Prerequisites(Python):

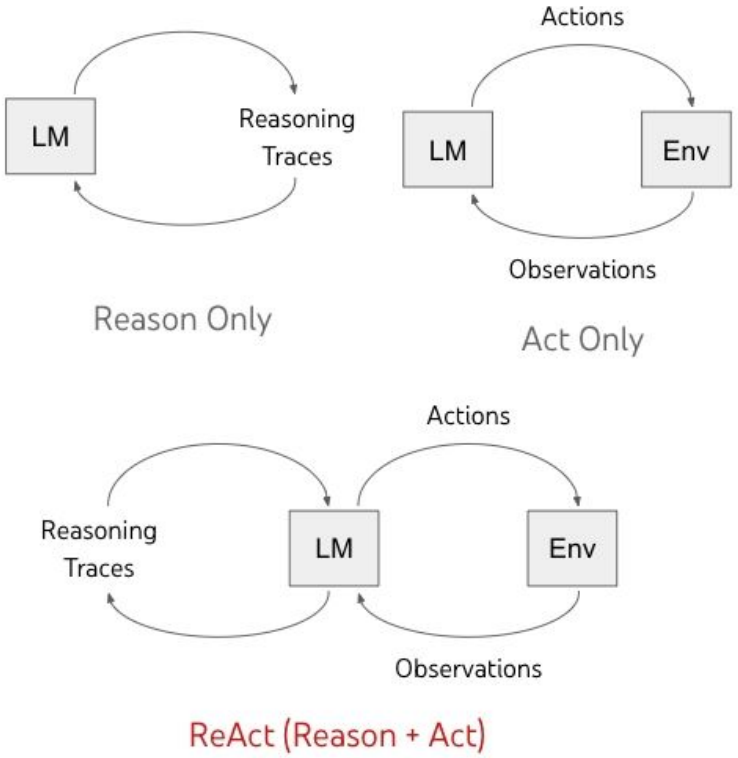
1. Flow control
2. Working with data structures like list and dictionaries.
3. Working with functions
4. Type annotations
5. Object-oriented programming(OOP)

Prerequisites(OpenAI and LangChain):

1. Managing environment variables by using a .env file.
2. Authenticating to LLM providers(such as OpenAI and anthropic)
3. Understanding the OpenAI and ChatCompletions API's.
4. Working with the prompt Templates and chains.
5. Output parsers.

REACT:Synergizing Reasoning and Acting in language model

- We will build the agent from scratch on the bases of ReAct.
-



PROJECT 1(Building a Simple ReAct Agent from Scratch)

- **Step 1:**
- This code snippet helps to load environment variables from .env file.

```
# loading the OpenAI API key
from dotenv import load_dotenv, find_dotenv
load_dotenv(find_dotenv(), override=True)
```

- find_dotenv():this function searches for the .env file ,typically starting from the current directory and moving upwards.
- load_dotenv():this function loads the key-value pair found in the .env file into the environment.

- **Step 2(optional):**
- A small snippet of code to creating a small chat .
- This chat returns a short and funny response .

```
from openai import OpenAI
client = OpenAI()

MODEL = 'gpt-4o-mini'
prompt = 'Write something short but funny.'

chat_completion = client.chat.completions.create(
    model=MODEL,
    messages=[{'role': 'user', 'content': prompt}]
)
```

```
chat_completion.choices[0].message.content
```

```
'Why did the scarecrow win an award? Because he was outstanding in his field!'
```

- **Step 2:**
- Creating a Agent class:
- Creating a Agent Class with three functions :-

```
class Agent:
    def __init__(self, system=''):
        self.system = system
        self.messages = []

        if self.system:
            self.messages.append({'role': 'system', 'content': system})

    def __call__(self, prompt):
        self.messages.append({'role': 'user', 'content': prompt})
        result = self.execute()
        self.messages.append({'role': 'assistant', 'content': result})
        return result

    def execute(self, model='gpt-4', temperature=0):
        completion = client.chat.completions.create(
                        model=model,
                        temperature=temperature,
                        messages=self.messages)

        return completion.choices[0].message.content
```

1. **__init__:** this is an constructor class .it Is initializing an agent object.

- **Self.system = system**: Stores an optic system message ,which is used to set an Initial behaviour or persona for AI.
- **self.message = []**: initializes an empty list to store the conversation history.
- **If self.system:self.messages.append({'role':'system','content':system}):** if a system message is provided,it's added to the messages list with the 'role':system.

2. **__call__(self,prompt)**:this method allows the agent object to be called like a function.

- **self.messages.append({'role':'user','content':'prompt'})**:Appends the user's prompt to the message list with the role 'user'.

- **result = self.execute():**Calls the execute function to send the messages to the language model to get a response.
- **self.messages.append({'role' : 'assistant','content':result}):**Appends to the language model's result to the messages list with the role 'assistant'. Then we return the result(the response of the llm

3. **execute(self,model='gpt-4',temperature=0):**this method handles the interaction with the llm API.

- **Completion = client.chat.completions.create(...):**this line send the conversation history (self.message),the specified model and temperature(controls the randomness of the output) to the language model API to generate a response.
- **return completion.choices[0].message.content:**extract and return the content of the first message generated by the language model.

- **Step 3:(Creating ReAct Prompt):**
- Defining a prompt to get structured output that loops through Thought,Actions,PAUSE,Observations.With example that specifies these steps.

- **STEP 4:(CREATING A TOOL):**
- Creating a tool step includes Defining a function that is used for calculation.
- In this code snippet we are defining a wikipedia function or method that interacts with the Wikipedia api to search for a given query q.

```python
# 3. the wikipedia() function uses the Wikipedia API to search for a specific query on Wikipedia
def wikipedia(q):
    response = httpx.get('https://en.wikipedia.org/w/api.php', params={
        'action': 'query',
        'list': 'search',
        'srsearch': q,
        'format': 'json'
    })
    results = response.json().get('query').get('search', [])

    if not results:
        return None
    return results[0]['snippet']
```

- **def wikipedia(q):** this defines a python function named wikipedia that takes one argument,q,which is here representing a search query.
- **response = httpx.get(.....)**: this lines makes an httpx get request to the wikipedia api endpoint . The parameter for the query request are :

1. **'action':'query'**:indicates a query action.
2. **'list':'search'**:specifies that the query should perform a search
3. **'srsearch':q** : provides the search term from the function's q argument.
4. **'format':'json'**: requests the response in JSON format.

- **response.json()**:converts the API response into a python dictionary.
- **.get('query').get('search',[])**:retrieves the 'search' list within the query section of JSON.

- **STEP 5:(TESTING THE AGENT):**
- **abot= Agent(prompt)**: initializing a variable my_agent with Agent class by passing prompt defined in step 3.
- query = "....":passing a query/question
- **next_prompt="...."**:it is getting the resu after wikipedia function is applied and used with the query as an argument .
- **print(next_prompt)**:printing the next_ prompt string ,which is now containing the observation .

```
1 abot = Agent(prompt)
```

```
1 query = '2024 United Kingdom elections'
2 abot(query)
```

```
'Thought: I should look up information about the 2024 United
ipedia: 2024 United Kingdom elections  \nPAUSE'
```

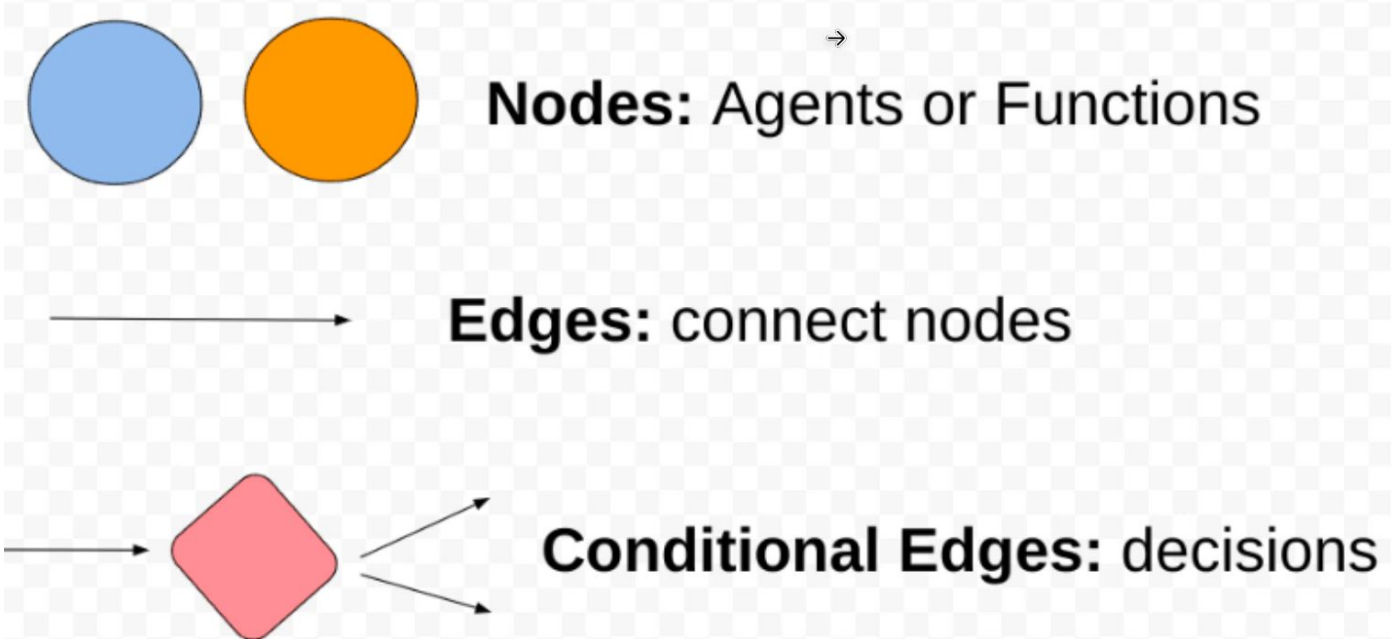
```
1 # calling the wikipedia() function and creating the next p
2 next_prompt = f'Observation: {wikipedia(query)}'
3 print(next_prompt)
```

- **FINAL STEP:(AUTOMATING THE AGENT):**
- The code and agent is automated by adding a function to maintain and print the result in a automated way.

LANGGRAPH : EXPLANATION

- Langgraph is an open-source framework(build by langchain) for creating stateful,multiagent system.
- Terminologies:

1. Graphs: represents the overall structure of the workflow,consisting of various interconnected nodes.



- A graph consists of nodes and edges.

2. Nodes: nodes in graph represents a discrete unit of work or a task that the agent needs to perform. Nodes are the agents of the function that are performed.
3. Edges: edges are connecting the nodes together on the basis of the requirement. There are two types of edges : normal edge and conditional edge.

>>Normal edge:are when one node should ALWAYS be called after another.

>>Conditional edge: are where a function is used to determine which node to go first.

- State machine: the state machine in langgraph in langgraph manages the state transitions of the workflow.
- It keeps the track of the current state of the agent and ensures that tasks are executed in the correct order ,based on the defined workflow.
- Langgraph creates these state machine by by specifying them as graph.

- **Cyclic graphs:** is simply a workflow graph where the execution can loop back to an earlier node instead of going forward to the next node.
- **“Human in the loop”(HITL):** is a paradigm where human intervention is integrated into automated workflows to enhance decision-making ,handles exception or provide feedback/insight.
- **Persistence:** refers to the ability to save the state of workflows ,allowing it to be resumed or recovered at a later time.

- **Flow engineering:** is a structured approach to software development where the steps are broken down in series of the well-defined steps and flows.
- Instead of using a single prompt to solve the problems ,flow engineering uses an interactive process that repeatedly runs and refines the generative result.

PROJECT 2: CHATBOT

- **STEP 1:**
- Importing the necessary files to be used.
- Langgraph imports:

1. Stategraph : for defining graph-based workflows
2. Annotated and typedict : for type hinting
3. add_messages: for managing message history within graph.

```python
from langgraph.graph import StateGraph
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI
```

```python
# loading the API key from .env
from dotenv import load_dotenv, find_dotenv
load_dotenv(find_dotenv(), override=True)
```

- LLM integrations:

1. ChatOpenAI from langchain_openai : this indicates the use of openai's language model within langgraph application.

- Environment setup: by using load_dotenv we can import the api keys from .env file.

- **STEP 2:(CREATING GRAPH):**

1. Setup llm model with api key.
2. Declare and define state class
3. Define chatbot function
4. Build the graph
5. Gradio chat interface
6. Launching gradio

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
```

```
# 1. Setup Groq LLM
llm = ChatGroq(
    groq_api_key=os.getenv("GROQ_API_KEY"),
    model="llama3-8b-8192"
)

# 2. Define the State type
from typing_extensions import TypedDict
from typing import List, Dict

class State(TypedDict):
    messages: List[Dict[str, str]]  # list of {"role":
"user"/"assistant", "content": "..."}

# 3. Define chatbot node
def chatbot(state: State) -> State:
    # Always ensure there's at least one user message
    if not state.get("messages") or not
state["messages"][-1]["content"].strip():
        return {"messages": state.get("messages", []) + [{"role":
"assistant", "content": "⚠️ Please enter a message."}]}

    # Pass conversation to Groq
    response = llm.invoke(state["messages"])
    return {"messages": state["messages"] + [{"role": "assistant",
"content": response.content}]}

# 4. Build graph
graph = StateGraph(State)
graph.add_node("chatbot", chatbot)
graph.add_edge(START, "chatbot")
graph.add_edge("chatbot", END)
app = graph.compile()

# 5. Gradio chat interface
def chat(user_input: str, history: List[Dict[str, str]]):
    # Combine history + new user input
    messages = history + [{"role": "user", "content": user_input}]
    result = app.invoke({"messages": messages})
    return result["messages"][-1]["content"]

gr.ChatInterface(chat, type="messages").launch()
```

- **STEP 3:(TAVILY AI):**
- Tavily ai is a search engine specifically optimized for large language model.
- If offers a powerful API designed to provide a factual,efficient and persistent search experience,specifically tailored for LLM's.
-

```python
from tavily import TavilyClient
import os

# initializing a Tavily client
client = TavilyClient(api_key=os.environ.get('TAVILY_API_KEY'))

response = client.search(query='What is the Bitcoin price today?')
response
```

🤖 Agentic Search

- **Definition:** A **search process managed by an AI agent** that can **reason, plan, and act** beyond just retrieving documents.

- **STEP 4:(ENHANCING THE PROJECT BY ADDING WEB SEARCH TOOL):**
- Adding tavily tool
- Importing TavilySearchTool from langchain_community.
- Then binding with the llm to use within the project for web searching

```
from langchain_community.tools.tavily_search import TavilySearchResults
tool = TavilySearchResults(max_results=3)
tools = [tool]
```

- **STEP 5:(ADDING MEMORY):**
- For adding memory we can use checkpoint memorySaver.

```
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.checkpoint.memory import MemorySaver

memory = SqliteSaver.from_conn_string(':memory:')
graph = graph_builder.compile(checkpointer=MemorySaver())
```

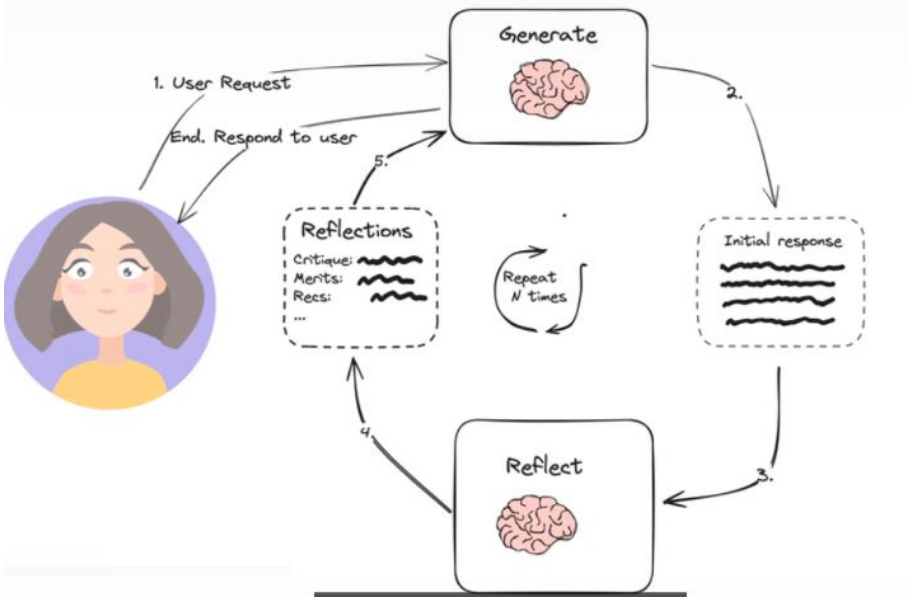
```
config = {'configurable': {'thread_id': '1'}}
```

```
prompt = 'Hi! My name is Andrei and you are a math tutor.'
events = graph.stream(
    {'messages': [('user', prompt)]}, config, stream_mode='values'
)
```

- For each thread_id it create a new session with different memory.

PROJECT-3(REFLECTION-TWEET GENERATOR)

- **REFLECTION:**
- IN LLM reflection refers to the process of prompting an LLM to observe its past steps(along with potential observation from tools/the environment) to assess the quality of the chosen action.
- It is used for things like re-planning,search,or evaluation.
- Reflection means getting the LLM to look back at its previous actions and feedback it received from tools or the environment.



Reflection means monitoring, evaluating, and adapting.

- **Monitor** means to continuously observe actions and outcomes.
- **Evaluate** means to assess the effectiveness of actions.
- **Adapt** means to modify strategies based on evaluations to improve future performance.

LANGSMITH

- Langsmith integrates with langchain and langgraph but also can functions independently.
- Langsmith streams the development lifecycle.
- Langsmith provides a unified platform for debugging,testing,evaluating,and monitoring LLM applications.
- Features of langsmith:-

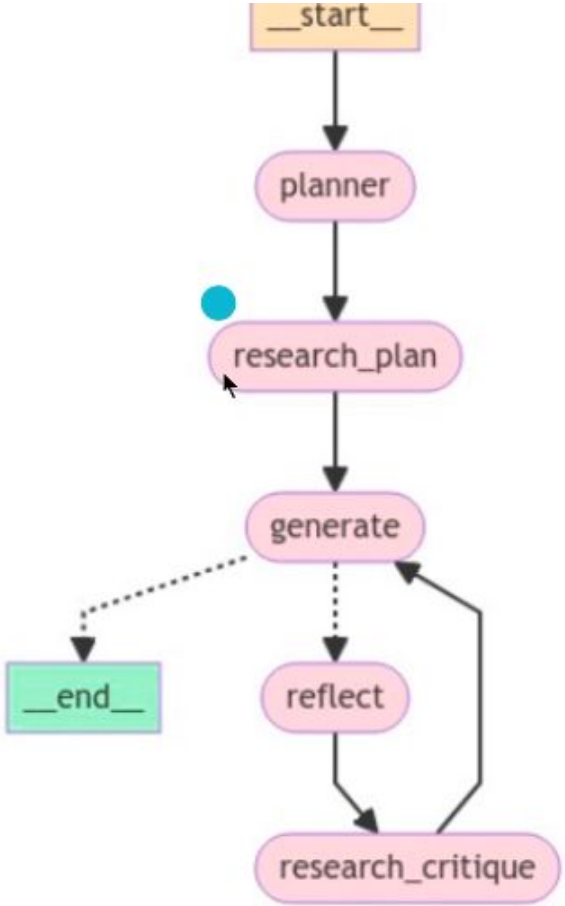
- **Tracing and Debugging:** LangSmith provides detailed trace logs of your application's runs. A "**run**" represents a unit of work or operation within your LLM application. These runs are grouped into **"traces"** that encapsulate the entire operation triggered by a user request.
- **Evaluation and Testing:** LangSmith allows developers to create datasets made up of inputs and expected outputs.
- **Monitoring and Observability:** LangSmith provides real-time monitoring capabilities. It tracks key performance metrics such as latency, cost, and user feedback scores, ensuring that any anomalies or performance issues are quickly identified and addressed.

- **Collaboration and Feedback:** LangSmith fosters collaboration among developers by providing features like annotation queues and feedback collection. These tools enable teams to add human insights and label traces, enhancing the debugging and evaluation processes.
- **Versioning and Comparison:** As applications evolve, it's important to compare different versions to ensure improvements or identify regressions. LangSmith offers a comparison view for side-by-side analysis of application versions, making it easier to track changes and their impacts on performance.

- Langsmith is used for tracing the workflow that is happening visually.

PROJECT 3(REFLEXION ESSAY WRITER)

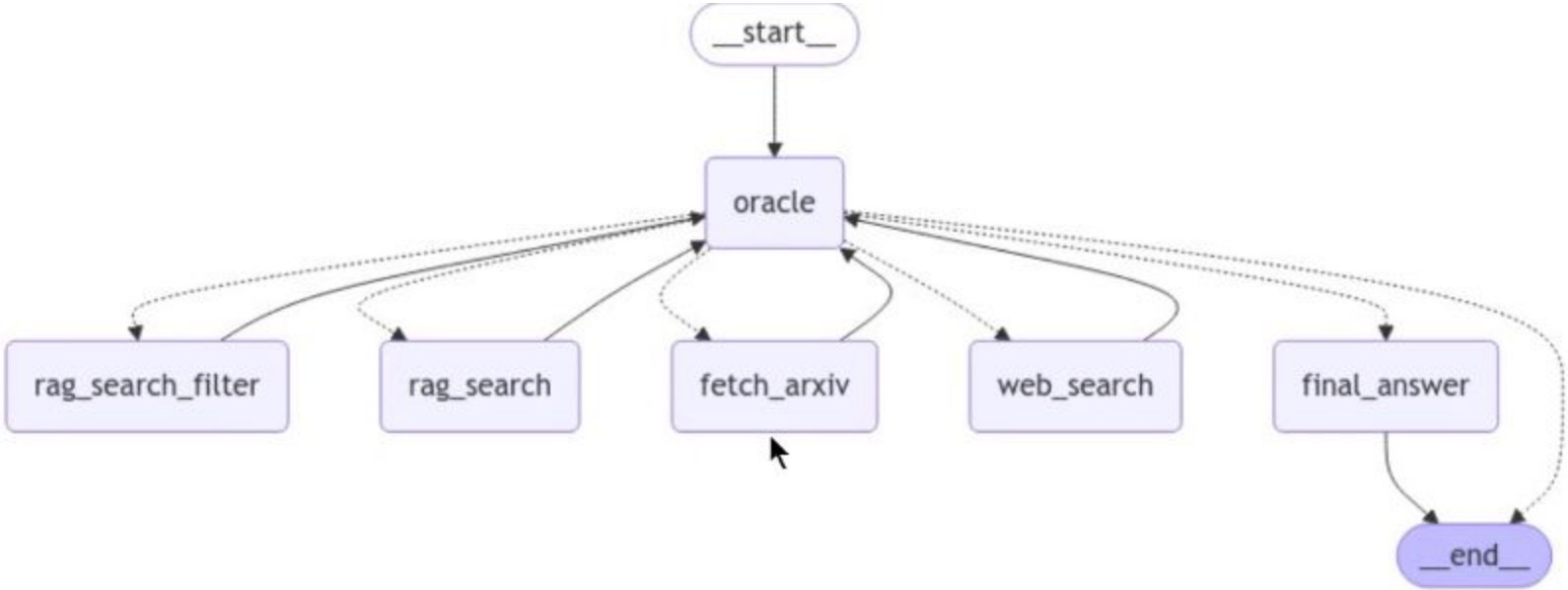
- Thai project is about creating an agent state that works in different steps along with reflexion.
- Where the first node is the planner node that plan what to do writer in the essay.
- Second node research_plan is going to carry out a research based on plan which involve s AI tc Retrieve relevant document.
- Third node is going to generate the essay.
- After writing the initial version of the essay, a conditional edge determines whether the essay is good or not it goes to reflect node .
- Then the research critique node generate the a critique of the current essay based on that the generate node generates essay again and again Same process happens.



FINAL PROJECT

- This project is about extracting pdf info and researching about information.
- The first step starts with extracting data from ArXiv into a pandas DataFrame.
- ArXiv is a platform where researcher generally publish their unreleased papers.
- The second step of is to download those research paper from the extracted dataframe.
- Then the third one is to load and splits those downloaded pdf files into chunks and then to put them into dataframe.
- After converting the pdf files into chunks we will use them for creating knowledge of RAG system by putting them on a vector database
- In this project we are using pinecone as the vector database for storing the vector embeddings.
- Then creating a tool for web searching by sing tavily AI
- building decision making pipeline.

1. Defining a agent state.
2. Defining the graph.
3. Generating reports
4. Building final research report.


__start__
oracle
rag_search_filter
rag_search
fetch_arxiv
web_search
final_answer
__end__